PHASE 2 – SYSTEM DESIGN

Step 1: System Architecture Design

1.1 Introduction

The Railway Reservation System will use a layered architecture. Each layer will perform a specific responsibility, making the system easier to develop, test, maintain and extend.

The architecture will separate the user interface, application logic, concurrency control and database operations.

1.2 Proposed Architecture

The system consists of four main layers:

1. Presentation Layer

- Provides the user interface.
- Displays available and booked seats.
- Allows passengers to select seats.
- Displays booking confirmation or rejection messages.
- Provides an administrative view of booking records.

2. Application Layer

- Receives booking requests.
- Validates passenger and seat information.
- Checks seat availability.
- Coordinates the booking process.
- Returns booking results to the interface.

3. Concurrency Control Layer

- Prevents simultaneous requests from allocating the same seat.
- Uses a locking mechanism during the critical seat-allocation operation.
- Ensures that the availability check and booking operation are performed safely.
- Handles requests when a seat has already been booked.

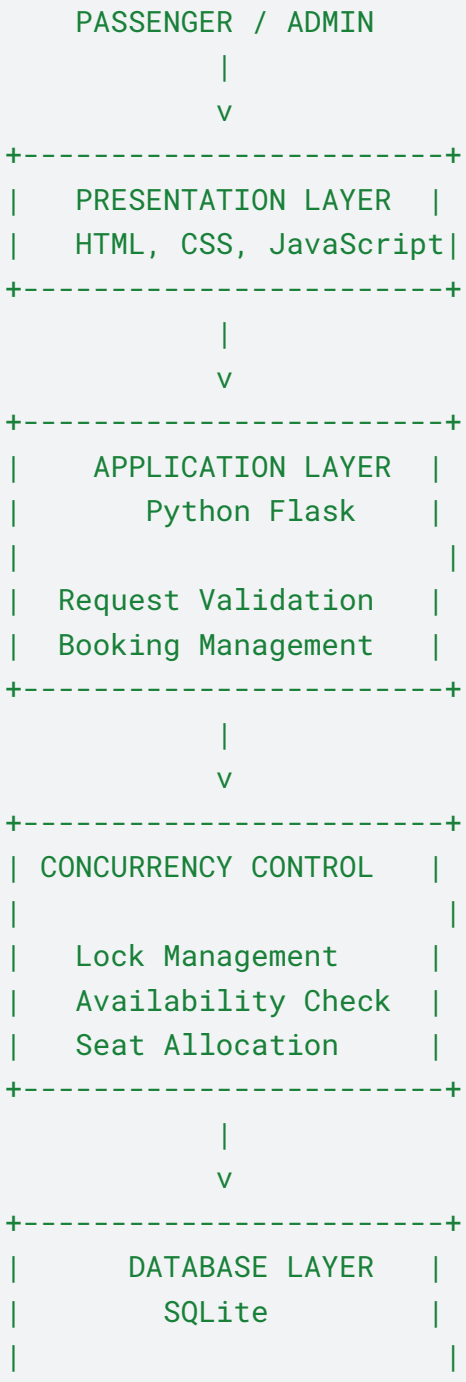
4. Database Layer

- Stores passenger information.

- Stores train journey and seat information.
- Maintains booking records.
- Updates seat availability.
- Enforces database constraints to prevent duplicate confirmed bookings.

1.3 Architecture Diagram

None



```
|  Passengers           |
|  Journeys              |
|  Seats                 |
|  Bookings              |
+------------------------+
```

1.4 Architecture Explanation

When a passenger selects a seat, the presentation layer sends a booking request to the Flask application. The application validates the request and passes it to the concurrency control layer.

The concurrency control layer ensures that two requests cannot successfully allocate the same seat. It performs the booking operation inside a database transaction.

The database stores the booking and updates the seat status. Finally, the application returns a success or failure response to the passenger.

1.5 Architectural Decision

A layered monolithic architecture is selected for the prototype because it is simpler to develop, test and demonstrate than a distributed microservices architecture.

For a production railway system handling large-scale traffic, the architecture could be extended using multiple application instances, a database designed for high concurrency, load balancing and distributed coordination.

Step 2: Technology Stack Finalisation

2.1 Selected Technologies

Component	Selected Technology	Purpose
Frontend	HTML	Structure of web pages
Styling	CSS	User interface design
Client-side logic	JavaScript	User interactions
Backend	Python	Application development

Web framework	Flask	Handling HTTP requests and APIs
Database	SQLite	Storing prototype data
Concurrency control	Python threading.Lock and database transactions	Protecting seat allocation
Testing	PyTest	Unit and integration testing
UI automation	Selenium	Automating booking workflows
Version control	Git	Tracking source-code changes
Repository hosting	GitHub	Storing and collaborating on code
CI/CD	Jenkins	Automating build, lint and test stages
Containerisation	Docker	Packaging the application
Static code analysis	SonarQube	Identifying code-quality issues

2.2 Justification for Technology Selection

Python: Python is selected because it has a simple syntax and provides libraries for application development, testing and concurrency.

Flask: Flask is lightweight and suitable for building the backend APIs required by the prototype.

SQLite: SQLite is selected because it is easy to configure and does not require a separate database server for the initial prototype.

Python threading.Lock: It demonstrates how concurrent threads can be prevented from entering the same critical section simultaneously.

PyTest: It allows automated testing of seat allocation, booking validation and concurrency behaviour.

Selenium: It automates the passenger booking workflow through the web interface.

Git and GitHub: They support source-code management, collaboration and merge-conflict exercises.

Jenkins: It automates the build and testing process.

Docker: It provides a consistent environment for running the application and database.

SonarQube: It helps identify bugs, vulnerabilities, code smells and code duplication.

2.3 Important Technology Limitation

The selected technologies are intended for an educational prototype.

A Python threading lock protects threads within one application process only. It does not automatically coordinate multiple application processes or multiple servers.

Similarly, SQLite is suitable for a small prototype but is not the recommended database choice for a production system handling 50,000 booking attempts per minute.

For production deployment, database-level transactions, unique constraints and suitable distributed coordination would be required. The production system would also need load testing and capacity planning.

Step 3: Database Design

3.1 Introduction

The database stores passenger details, journey information, seat information and booking records.

The database must maintain consistency and prevent multiple confirmed bookings for the same seat on the same journey.

3.2 Proposed Database Tables

Table 1: Passengers

Attribute	Data Type	Description
passenger_id	INTEGER	Primary key
passenger_name	VARCHAR	Passenger name
email	VARCHAR	Passenger email

phone	VARCHAR	Passenger contact number

Primary Key: passenger_id

Table 2: Journeys

Attribute	Data Type	Description
journey_id	INTEGER	Primary key
train_number	VARCHAR	Train identification number
source	VARCHAR	Starting station
destination	VARCHAR	Destination station
journey_date	DATE	Date of journey

Primary Key: journey_id

Table 3: Seats

Attribute	Data Type	Description
seat_id	INTEGER	Primary key
journey_id	INTEGER	Foreign key
seat_number	VARCHAR	Seat identification
seat_type	VARCHAR	Seat category
status	VARCHAR	Available or booked

Primary Key: seat_id

Foreign Key: journey_id references Journeys(journey_id).

Table 4: Bookings

Attribute	Data Type	Description
booking_id	INTEGER	Primary key
passenger_id	INTEGER	Foreign key
seat_id	INTEGER	Foreign key

journey_id	INTEGER	Foreign key
booking_status	VARCHAR	Confirmed or rejected
booking_time	DATETIME	Booking timestamp

Primary Key: booking_id

Foreign Keys:

- passenger_id references Passengers(passenger_id).
- seat_id references Seats(seat_id).
- journey_id references Journeys(journey_id).

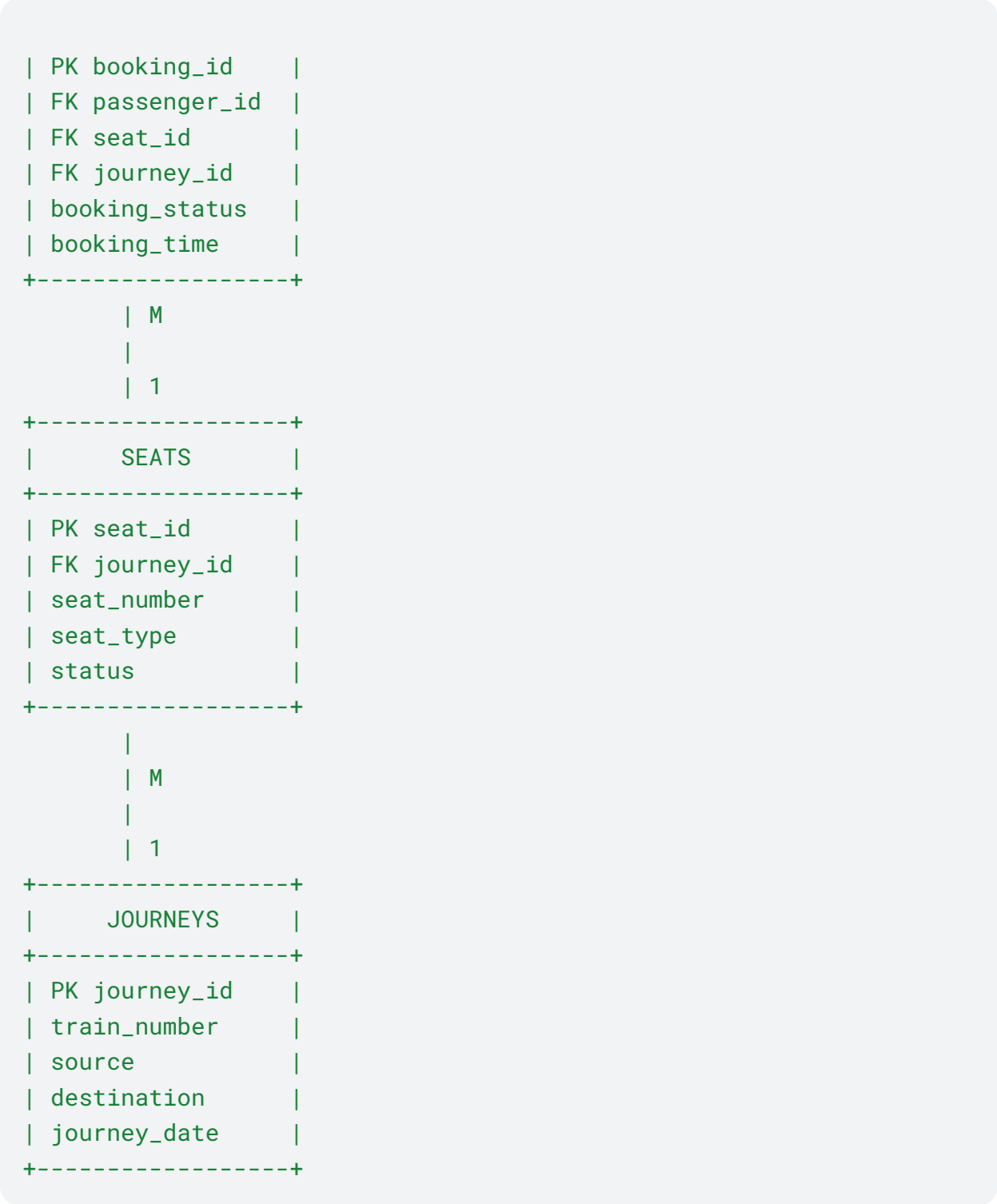
3.3 Database Relationships

1. One passenger can make multiple bookings.
2. One journey can contain multiple seats.
3. One journey can have multiple booking records.
4. Each booking belongs to one passenger.
5. Each booking refers to one seat on one journey.

3.4 ER Diagram

None

```
+------------------+
|    PASSENGERS    |
+------------------+
| PK passenger_id  |
| passenger_name   |
| email            |
| phone            |
+------------------+
         |
         | 1
         |
         | M
+------------------+
|     BOOKINGS     |
+------------------+
```



3.5 Database Integrity Rules

The following rules will be applied:

1. Every passenger must have a unique passenger ID.
2. Every journey must have a unique journey ID.
3. Every seat must belong to a valid journey.
4. Every booking must reference an existing passenger, journey and seat.
5. A seat cannot have more than one confirmed booking for the same journey.
6. A rejected booking must not change the seat's confirmed booking status.

A database-level uniqueness constraint should enforce the fifth rule.

For example, a unique index on `(journey_id, seat_id)` for confirmed bookings can prevent duplicate confirmed records.

The booking operation must also use a transaction so that checking availability and creating the booking cannot become inconsistent.

Step 4: UML Use-Case Diagram

4.1 Introduction

A use-case diagram represents the interaction between users and the system. It identifies the major functions available to passengers and administrators.

4.2 Actors

1. Passenger

- Views available seats.
- Selects a seat.
- Submits a booking request.
- Receives booking confirmation.
- Views booking history.

2. Administrator

- Views booking records.
- Monitors booking activity.
- Checks seat availability.
- Monitors system performance.

4.3 Use-Case Diagram

None



4.4 Use-Case Description

Use Case ID	Use Case	Actor	Description

UC-01	View Seats	Passenger	Displays available and booked seats
UC-02	Select Seat	Passenger	Allows selection of an available seat
UC-03	Book Seat	Passenger	Submits a booking request
UC-04	Validate Availability	System	Checks whether the seat is available
UC-05	Allocate Seat	System	Allocates the seat using concurrency control
UC-06	Confirm Booking	System	Returns booking confirmation
UC-07	View History	Passenger	Displays previous booking records
UC-08	View Records	Administrator	Displays booking information
UC-09	Monitor System	Administrator	Monitors booking activity and performance

4.5 Main Use-Case Flow: Book a Seat

1. Passenger opens the reservation system.
2. Passenger views available seats.
3. Passenger selects a seat.
4. Passenger submits a booking request.
5. System validates the request.
6. System acquires the required concurrency lock.
7. System checks seat availability inside a database transaction.
8. If the seat is available, the system creates the confirmed booking.
9. The system updates the seat status.
10. The system commits the transaction and releases the lock.
11. Passenger receives booking confirmation.

If the seat is already booked, the system rejects the request and informs the passenger.

Step 5: UML Class Diagram

5.1 Introduction

A class diagram represents the static structure of the system. It shows the classes, their attributes, methods and relationships.

5.2 Main Classes

1. Passenger

- passenger_id
- passenger_name
- email
- view_available_seats()
- make_booking()
- view_booking_history()

2. Journey

- journey_id
- train_number
- source
- destination
- journey_date
- get_journey_details()

3. Seat

- seat_id
- seat_number
- seat_type
- status
- check_availability()
- update_status()

4. Booking

- booking_id
- passenger_id
- seat_id
- journey_id
- booking_status
- booking_time
- create_booking()
- cancel_or_reject_booking()

5. BookingService

- validate_request()
- process_booking()
- confirm_booking()
- reject_booking()

6. LockManager

- acquire_lock()
- release_lock()

7. DatabaseManager

- connect_database()
- save_booking()
- update_seat()
- retrieve_booking()

5.3 Class Diagram

None




| create_booking()
| reject_booking()
Seat
seat_id
seat_number
seat_type
status
check_
availability()
update_
status()
Journey
journey_id
train_number
source
destination
journey_date
BookingService
validate_request()
process_booking()
confirm_booking()
reject_booking()
LockManager
acquire_lock()
release_lock()

```
          |
          v
+----------------------+
|    DatabaseManager   |
+----------------------+
| connect_database()   |
| save_booking()       |
| update_seat()        |
| retrieve_booking()   |
+----------------------+
```

5.4 Class Relationships

- Passenger can have multiple bookings.
- Each booking belongs to one passenger.
- Each booking refers to one seat and one journey.
- BookingService coordinates the booking process.
- LockManager controls access to the critical booking operation.
- DatabaseManager performs database operations.

Step 6: Component Design and Data Flow

6.1 Component Design

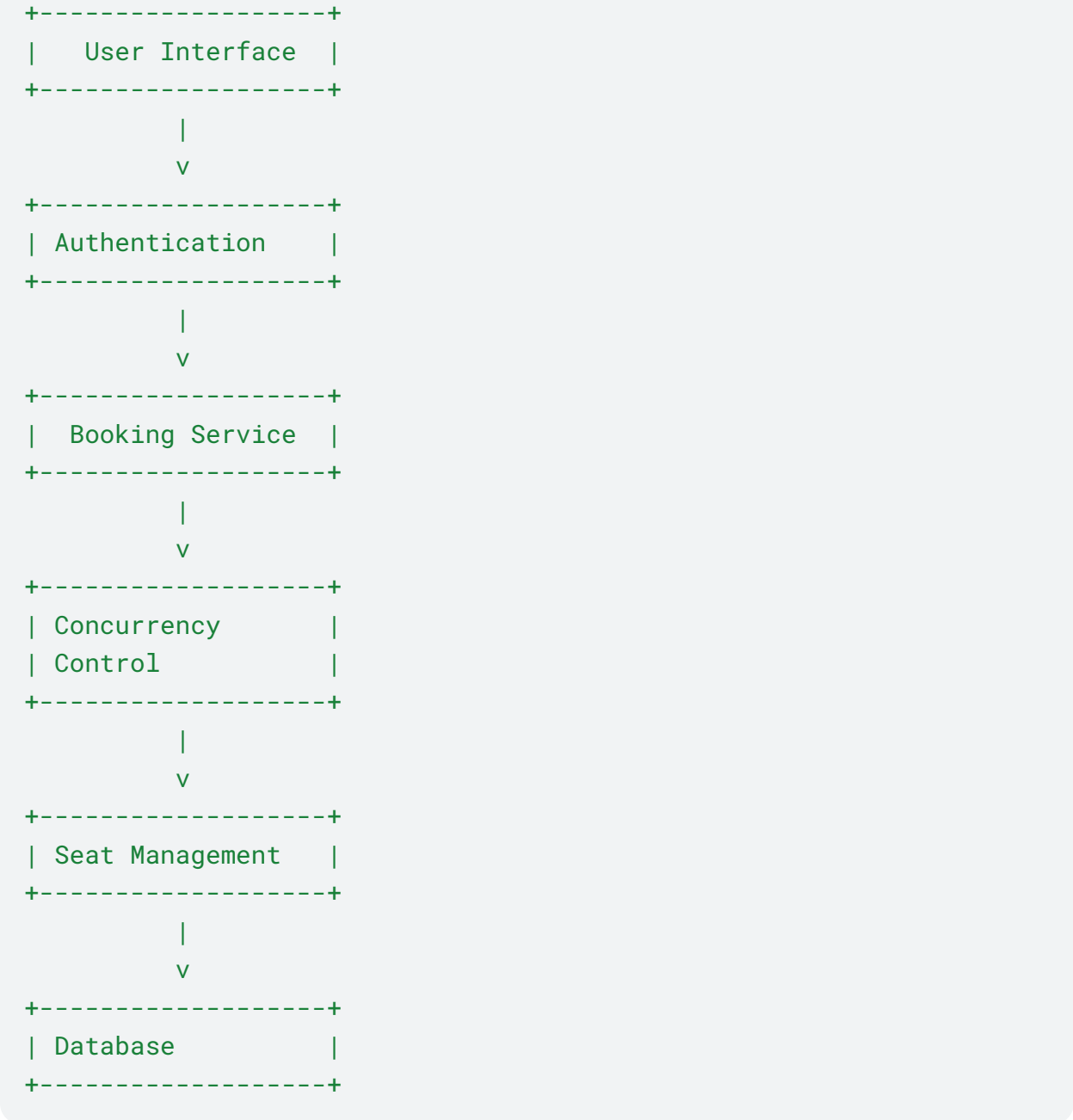
The system will be divided into the following components:

Component	Responsibility
User Interface	Displays seats and booking information
Authentication Component	Validates user identity
Seat Management Component	Retrieves and updates seat information
Booking Component	Processes booking requests
Concurrency Component	Controls simultaneous seat allocation

Database Component	Stores and retrieves records
Admin Component	Displays booking and monitoring information

6.2 Component Interaction

None



6.3 Data Flow of Seat Booking

Input:

- Passenger ID
- Journey ID
- Seat ID

Processing:

1. Receive booking request.
2. Validate passenger and journey details.
3. Acquire the concurrency lock.
4. Begin a database transaction.
5. Check whether the seat is available.
6. Create the booking record.
7. Update the seat status.
8. Commit the transaction.
9. Release the lock.

Output:

- Booking successful message, or
- Seat unavailable message.

6.4 Data Flow Diagram

None

```
       +------------+
       | Passenger  |
       +------------+
             |
             | Booking Request
             v
   +--------------------+
   | Booking Management |
   +--------------------+
             |
             v
   +--------------------+
   | Concurrency Control|
   +--------------------+
             |
             v
```



6.5 Concurrent Booking Example

Assume only one seat remains available.

Two passengers, A and B, submit requests for the same seat at almost the same time.

1. Passenger A submits a booking request.
2. Passenger B submits a booking request.
3. The concurrency control mechanism ensures that the booking operations are safely serialized.
4. Passenger A obtains the required lock first.
5. The system checks availability and confirms Passenger A's booking.
6. The database transaction commits.
7. The lock is released.
8. Passenger B's request is processed.
9. The system finds that the seat is already booked.
10. Passenger B receives a seat-unavailable response.

Expected result: Only one confirmed booking exists for the seat and journey.

The order of successful booking is determined by which transaction safely completes first; the system must not promise that the earliest network request always wins.

Step 7: Booking API Design

7.1 Introduction

The backend will expose REST API endpoints so that the frontend can communicate with the Flask application.

7.2 Proposed API Endpoints

Method	Endpoint	Purpose
GET	/seats	Retrieve all seats
GET	/seats/available	Retrieve available seats
GET	/journeys	Retrieve journey information
POST	/bookings	Submit a booking request
GET	/bookings/{id}	Retrieve a booking
GET	/passengers/{id}/bookings	Retrieve passenger booking history
GET	/admin/bookings	View all booking records
GET	/admin/metrics	Retrieve system monitoring metrics

7.3 Example Booking Request

```JSON
{
    "passenger_id": 101,
    "journey_id": 501,
    "seat_id": 25
}
```

7.4 Example Successful Response

```JSON
{
    "status": "success",
    "message": "Seat booked successfully",
    "booking_id": 1001,
    "seat_id": 25
}
```

7.5 Example Rejected Response

```JSON
{
    "status": "failed",
    "message": "Seat is already booked"
}
```

7.6 HTTP Status Codes

Status Code	Meaning
200	Request successful
201	Booking created
400	Invalid request
401	Authentication required
404	Resource not found
409	Seat already booked or booking conflict
500	Internal server error

The booking endpoint should be designed to handle repeated client requests safely. An idempotency key can be introduced to avoid creating duplicate bookings when a passenger retries after a timeout.

Step 8: Concurrency Control Design

8.1 Introduction

Concurrency control is the most important part of this project because multiple passengers may try to book the same seat simultaneously.

Without proper synchronization, a race condition can occur.

8.2 Race Condition

A race condition occurs when two or more operations access and modify shared data at the same time, causing an incorrect result.

Example:

```
None
Passenger A checks seat availability.
Passenger B checks seat availability.

Both see the seat as AVAILABLE.

Passenger A books the seat.
Passenger B also books the same seat.

Result: Double booking.
```

8.3 Proposed Solution

The prototype will use:

- A lock around the critical booking operation.
- A database transaction.
- A conditional seat-status update.
- A database uniqueness constraint.

The lock prevents concurrent threads in the same application process from executing the protected operation simultaneously.

The database transaction ensures that the booking record and seat status are updated consistently.

The database constraint provides an additional safeguard against duplicate confirmed bookings.

8.4 Proposed Booking Algorithm

```
None
START

Receive booking request

Validate passenger and seat details

Acquire concurrency lock

BEGIN DATABASE TRANSACTION

Check seat availability

IF seat is AVAILABLE:

    Create booking record

    Update seat status to BOOKED

    COMMIT TRANSACTION

    Return BOOKING SUCCESSFUL

ELSE:

    ROLLBACK TRANSACTION

    Return SEAT UNAVAILABLE

Release concurrency lock

END
```

8.5 Important Design Consideration

A process-level threading lock is sufficient only for coordinating threads within one running application process.

If multiple Flask workers or multiple servers are used, they will not share the same in-memory lock. In that situation, database-level coordination must enforce correctness across all workers.

For the prototype, the database transaction and uniqueness constraint are essential safeguards even when a Python lock is used.

Step 9: Security and Error-Handling Design

9.1 Security Requirements

The system will follow basic security practices to protect passenger and booking information.

Security Area	Proposed Design
Authentication	Validate passenger login
Authorisation	Restrict administrator functions
Password Security	Store password hashes, never plain-text passwords
Input Validation	Validate passenger, journey and seat IDs
SQL Injection	Use parameterised database queries
Data Protection	Avoid exposing unnecessary passenger information
Session Security	Use secure session configuration
Error Handling	Return controlled error messages
Logging	Record booking events and failures without sensitive data

9.2 Error Handling

The system should handle the following situations:

Error ID	Situation	Expected System Behaviour
ERR-01	Invalid passenger ID	Reject the request
ERR-02	Invalid seat ID	Display an error

ERR-03	Seat already booked	Reject the booking
ERR-04	Database failure	Roll back the transaction
ERR-05	Lock acquisition timeout	Return a temporary failure
ERR-06	Invalid request data	Return validation error
ERR-07	Duplicate request	Prevent duplicate booking
ERR-08	Unexpected server error	Log the issue and return a safe error response

9.3 Logging Design

The system should record:

- Booking request received.
- Booking request accepted.
- Booking request rejected.
- Seat allocation completed.
- Database transaction failed.
- Concurrency-related conflict.
- Request processing duration.

Logs must not contain passwords, authentication tokens or unnecessary personal information.

Step 10: System Design Validation and Traceability

10.1 Requirement-to-Design Mapping

This table ensures that the requirements from Phase 1 are represented in the system design.

Requirement ID	Requirement	Design Component
FR-01	Display available seats	Seat Management
FR-02	Select an available seat	User Interface
FR-03	Accept multiple booking requests	Booking Service
FR-04	Verify seat availability	Seat Management

FR-05	Apply concurrency control	Lock Manager
FR-06	Prevent double booking	Transactions and unique constraint
FR-07	Update seat status	Database Manager
FR-08	Confirm successful booking	Booking Service
FR-09	Reject unavailable seats	Booking Service
FR-10	Maintain booking records	Database Layer
NFR-01	95% of bookings under 2 seconds	Performance Monitoring
NFR-02	Evaluate 50,000 attempts per minute	Load Testing
NFR-03	Zero duplicate confirmed bookings	Concurrency Testing
NFR-04	Maintain consistent records	Database Transactions
NFR-05	Protect booking information	Security Layer
NFR-06	Maintain modular code	Layered Architecture
NFR-07	Maintain availability during peak demand	System Architecture and Load Testing
NFR-08	Provide a simple interface	Presentation Layer

10.2 Design Validation Checklist

No.	Validation Point	Expected Result
1	All functional requirements mapped	Complete
2	All non-functional requirements mapped	Complete
3	Database relationships defined	Complete
4	Primary and foreign keys identified	Complete
5	Duplicate booking prevention designed	Complete
6	Concurrency control mechanism defined	Complete
7	API endpoints identified	Complete

8	User interactions documented	Complete
9	Security and error handling considered	Complete
10	Design supports future testing	Complete

10.3 Phase 2 Deliverables

The following design documents have been prepared:

1. System architecture diagram.
2. Technology stack selection and justification.
3. Database table design.
4. Entity Relationship Diagram.
5. UML use-case diagram.
6. UML class diagram.
7. Component design.
8. Data Flow Diagram.
9. REST API design.
10. Concurrency control algorithm.
11. Security and error-handling design.
12. Requirement-to-design traceability table.

10.4 Conclusion

Phase 2 converts the requirements of the Railway Reservation System into a technical design. A layered architecture using Python Flask, SQLite, HTML, CSS and JavaScript has been proposed for the prototype.

The database design maintains relationships between passengers, journeys, seats and bookings. The concurrency control design combines a lock, database transactions and uniqueness constraints to prevent duplicate confirmed bookings.

The architecture is suitable as a foundation for the educational prototype. However, the 50,000-attempts-per-minute target requires a separate production-scale architecture and performance evaluation.

The next phase will use these designs to prepare detailed implementation specifications, database scripts, module-level logic and the project structure.